

SPOF + 2° Haproxy

Henrio Chambal BTS SIO SSIR B

Introduction

Dans le cadre de ce travail, l'objectif était d'assurer la tolérance aux pannes du service web de StadiumCompany en supprimant le point unique de défaillance (SPOF) lié au premier serveur HAProxy.

Pour cela, une seconde machine HAProxy a été mise en place et configurée en haute disponibilité à l'aide du service Heartbeat, afin que l'un prenne automatiquement le relais de l'autre en cas de panne.

Les deux serveurs HAProxy sont reliés au même réseau que les serveurs web et la machine cliente, et partagent une adresse IP virtuelle qui garantit la continuité du service sans interruption.

Cette mise en place permet d'obtenir une infrastructure redondante et résiliente, capable d'assurer la disponibilité du site web même en cas de défaillance d'un des nœuds du cluster.

Création du clone de la machine HAProxy

Cette étape consiste à créer un clone complet (Full Clone) du serveur HAProxy principal afin de disposer d'un second nœud identique dans le cluster.

Le clone complet permet d'obtenir une copie totalement indépendante du système d'origine, contrairement au "linked clone" qui dépend du disque de la machine source. Cette opération est essentielle pour éliminer le SPOF (Single Point of Failure), c'est-à-dire éviter qu'une panne du premier HAProxy rende le service indisponible.

Clone Virtual Machine Wizard

Clone Type
How do you want to clone this virtual machine?

Clone method

☐ Create a linked clone
A linked clone is a reference to the original virtual machine and requires less disk space to store. However, it cannot run without access to the original virtual machine.

☒ Create a full clone
A full clone is a complete copy of the original virtual machine at its current state. This virtual machine is fully independent, but requires more disk space to store.

< Précédent Suivant > Annuler

Configuration réseau du second HAProxy

Après la création du clone, il est indispensable de modifier l'adresse IP pour éviter tout conflit avec le HAProxy principal.

Dans ce fichier `/etc/network/interfaces`, on assigne au clone l'adresse 172.20.0.12 sur l'interface `ens33`, tandis que l'interface `ens34` reste configurée en DHCP.

Cette étape permet au second HAProxy d'être reconnu comme un nœud indépendant sur le réseau local, tout en partageant la même configuration logicielle que le premier.

```
GNU nano 7.2 /etc/network/interfaces *
auto lo
iface lo inet loopback

auto ens33
iface ens33 inet static
    address 172.20.0.12_
    netmask 255.255.255.0

auto ens34
iface ens34 inet dhcp
```

Redémarrage du réseau et changement du nom d'hôte

Après avoir modifié l'adresse IP du clone, il faut redémarrer le service réseau avec la commande `systemctl restart networking` afin que les nouveaux paramètres soient pris en compte.

Ensuite, on change le nom d'hôte de la machine avec `hostnamectl set-hostname HAProxy2`, ce qui permet d'identifier clairement le second nœud dans le cluster.

Cette distinction entre *HAProxy* et *HAProxy2* est essentielle pour la communication et la synchronisation entre les deux serveurs Heartbeat.

```
root@HAProxy:~# systemctl restart networking
root@HAProxy:~# hostnamectl set-hostname HAProxy2
root@HAProxy:~#
```

Installation de Heartbeat sur les deux serveurs HAProxy

Une fois les deux machines HAProxy configurées avec leurs adresses IP et noms distincts, on procède à l'installation du service Heartbeat à l'aide de la commande :

```
apt install -y heartbeat
```

Cette étape permet de mettre en place le mécanisme de haute disponibilité (HA) entre les deux nœuds.

Heartbeat est le service responsable de surveiller l'état de chaque serveur et d'assurer un basculement automatique du trafic en cas de panne de l'un d'eux, garantissant ainsi

la continuité du service du load balancer HAProxy.

```
root@HAProxy:~# apt update
apt install -y heartbeat
Atteint :1 http://deb.debian.org/debian bookworm InRelease
Réception de :2 http://deb.debian.org/debian bookworm-updates InRelease [55,4 kB]
Réception de :3 http://security.debian.org/debian-security bookworm-security InRelease [48,0 kB]
Réception de :4 http://security.debian.org/debian-security bookworm-security/main Sources [180 kB]
Réception de :5 http://security.debian.org/debian-security bookworm-security/main amd64 Packages [284 kB]
567 ko réceptionnés en 1s (869 ko/s)
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
```

Configuration du fichier ha.cf

Dans cette étape, on crée et configure le fichier `/etc/ha.d/ha.cf`, qui définit les paramètres de communication entre les deux nœuds du cluster Heartbeat.

Les directives utilisées permettent de :

- `logfile /var/log/heartbeat` : enregistrer les événements liés au service.
- `keepalive` et `deadtime` : définir la fréquence d'envoi des signaux et le délai avant de considérer un nœud comme inactif.
- `bcast ens33` : indiquer l'interface réseau utilisée pour la communication entre les serveurs HAProxy.
- `node HAProxy` et `node HAProxy2` : lister les deux machines du cluster.
- `auto_failback on` : permettre le retour automatique du service principal une fois rétabli.

Ce fichier est essentiel pour la synchronisation entre les deux HAProxy et le bon fonctionnement du mécanisme de haute disponibilité (HA).

```
GNU nano 7.2 /etc/ha.d/ha.cf *
logfile /var/log/heartbeat
logfacility local0
keepalive 2
deadtime 10
bcast ens33
node HAProxy
node HAProxy2
auto_failback on
EOF
```

Configuration du fichier authkeys

Dans ce fichier, on définit le mécanisme d'authentification utilisé par Heartbeat pour sécuriser les échanges entre les deux serveurs HAProxy.

- `auth 1` indique la méthode d'authentification à utiliser.
- `1 md5 greta` précise que la clé numéro 1 utilise le chiffrement MD5 avec comme mot de passe *greta*.
- La commande `chmod 600 /etc/ha.d/authkeys` restreint ensuite les droits d'accès au fichier pour que seul l'utilisateur root puisse le lire ou le modifier, garantissant la sécurité de la clé.

Cette étape est indispensable pour éviter toute communication non autorisée entre les deux nœuds du cluster Heartbeat.

```
GNU nano 7.2 /etc/ha.d/authkeys *
auth 1
1 md5 greta
EOF
chmod 600 /etc/ha.d/authkeys
```

Correction du fichier ha.cf

Dans cette étape, on a corrigé le fichier de configuration principal de Heartbeat (/etc/ha.d/ha.cf).

L'erreur précédente provenait de la présence du mot-clé EOF, qui n'a pas lieu d'être dans ce fichier et empêchait le service de démarrer correctement.

Après suppression de cette ligne, le fichier contient uniquement les paramètres essentiels :

- logfile et logfacility définissent la gestion des logs,
- keepalive et deadtime précisent les intervalles d'échange entre les nœuds,
- bcast ens33 indique l'interface réseau utilisée pour la communication Heartbeat,
- node HAProxy et node HAProxy2 listent les serveurs du cluster,
- auto_failback on permet de rétablir automatiquement le nœud principal lorsqu'il redevient disponible.

Cette correction a permis à Heartbeat d'interpréter correctement la configuration pour lancer la synchronisation entre les deux serveurs.

```
GNU nano 7.2 /etc/ha.d/ha.cf *
logfile /var/log/heartbeat
logfacility local0
keepalive 2
deadtime 10
bcast ens33
node HAProxy
node HAProxy2
auto_failback on
```

Attribution des droits root sur Heartbeat

Après avoir configuré le fichier authkeys, il est indispensable de sécuriser l'accès à ce fichier, car il contient la clé d'authentification utilisée entre les deux serveurs du cluster. Les commandes chmod 600 et chown root:root ont donc été appliquées pour :

- Restreindre la lecture et l'écriture du fichier uniquement à l'utilisateur root,
- Empêcher tout autre utilisateur d'accéder au contenu, renforçant ainsi la sécurité du cluster.

Ces permissions correctes sont nécessaires pour que le service Heartbeat démarre sans erreur d'authentification.

```
GNU nano 7.2 /etc/ha.d/authkeys *
chmod 600 /etc/ha.d/authkeys
chown root:root /etc/ha.d/authkeys
```

Vérification du service Heartbeat sur HAProxy

Après avoir configuré les fichiers ha.cf, authkeys et défini les bons droits, le service Heartbeat a été redémarré afin de valider la configuration.

La commande `systemctl status heartbeat` permet de vérifier son état.

Le statut `active (running)` indique que le cluster est bien opérationnel et que la communication entre les nœuds est fonctionnelle.

Le log affiche également que le lien réseau utilisé pour la synchronisation (ens34) est bien actif et que le service Heartbeat a correctement démarré, assurant ainsi la redondance entre les deux serveurs HAProxy

```
root@HAProxy:~# systemctl stop heartbeat
root@HAProxy:~# systemctl start heartbeat
root@HAProxy:~# systemctl status heartbeat
● heartbeat.service - Heartbeat High Availability Cluster Communication and Membership
   Loaded: loaded (/lib/systemd/system/heartbeat.service; enabled; preset: enabled)
   Active: active (running) since Sun 2025-11-09 21:41:42 CET; 5s ago
     Docs: man:heartbeat(8)
           http://www.linux-ha.org/wiki/Documentation
  Main PID: 32772 (heartbeat)
    Tasks: 4 (limit: 2272)
   Memory: 7.2M
      CPU: 27ms
   CGroup: /system.slice/heartbeat.service
           └─32772 "heartbeat: master control process"
             └─32775 "heartbeat: FIFO reader"
               └─32776 "heartbeat: write: bcast ens34"
                 └─32777 "heartbeat: read: bcast ens34"

nov. 09 21:41:42 HAProxy heartbeat[32772]: [32772]: info: Configuration validated. Starting heartbeat
nov. 09 21:41:42 HAProxy heartbeat[32772]: [32772]: info: heartbeat: version 3.0.6
nov. 09 21:41:42 HAProxy heartbeat[32772]: [32772]: WARN: No Previous generation - starting at 176272
nov. 09 21:41:42 HAProxy heartbeat[32772]: [32772]: info: Heartbeat generation: 1762720903
nov. 09 21:41:42 HAProxy heartbeat[32772]: [32772]: info: No uuid found for current node - generating
nov. 09 21:41:42 HAProxy heartbeat[32772]: [32772]: info: Creating FIFO /var/lib/heartbeat/fifo.
nov. 09 21:41:42 HAProxy heartbeat[32772]: [32772]: info: glib: UDP Broadcast heartbeat started on po
nov. 09 21:41:42 HAProxy heartbeat[32772]: [32772]: info: glib: UDP Broadcast heartbeat closed on po
nov. 09 21:41:42 HAProxy heartbeat[32772]: [32772]: info: Local status now set to: 'up'
nov. 09 21:41:42 HAProxy heartbeat[32772]: [32772]: info: Link haproxy:ens34 up.
lines 1-25/25 (END)
```

Vérification de la configuration HAProxy

Dans cette étape, le fichier de configuration principal de HAProxy, situé dans `/etc/haproxy/haproxy.cfg`, a été vérifié à l'aide de la commande :

`haproxy -c /etc/haproxy/haproxy.cfg`.

Le message "Configuration file is valid" confirme que la syntaxe et les paramètres du fichier sont corrects sur les deux nœuds (HAProxy1 et HAProxy2).

Cette validation est essentielle avant le démarrage du service, afin d'éviter toute erreur

de configuration qui pourrait empêcher la mise en place du load balancing ou du basculement automatique.

```
root@HAProxy:~# nano /etc/haproxy/haproxy.cfg
root@HAProxy:~# nano /etc/haproxy/haproxy.cfg^C
root@HAProxy:~# haproxy -c -f /etc/haproxy/haproxy.cfg
Configuration file is valid
root@HAProxy:~# ^C
root@HAProxy:~# █
```

Vérification du bon fonctionnement des serveurs web

Cette étape consiste à vérifier que les serveurs web reliés au cluster HAProxy sont bien accessibles et répondent correctement.

Pour cela, la commande `curl -sI http://172.20.0.11` et `curl -sI http://172.20.0.12` a été utilisée sur le premier nœud HAProxy.

La réponse HTTP/1.1 200 OK confirme que les deux serveurs web (SRV-WEB1 et SRV-WEB2) sont opérationnels et renvoient une réponse valide.

Cela garantit que HAProxy pourra distribuer la charge entre eux sans erreur lors du basculement ou du load balancing.

```
root@HAProxy:~# apt update && apt install -y curl # si curl absent
curl -sI http://172.20.0.11 | head -1
curl -sI http://172.20.0.12 | head -1
Atteint :1 http://deb.debian.org/debian bookworm InRelease
Atteint :2 http://deb.debian.org/debian bookworm-updates InRelease
Atteint :3 http://security.debian.org/debian-security bookworm-security InRelease
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Tous les paquets sont à jour.
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
curl est déjà la version la plus récente (7.88.1-10+deb12u14).
0 mis à jour, 0 nouvellement installés, 0 à enlever et 0 non mis à jour.
HTTP/1.1 200 OK
HTTP/1.1 200 OK
root@HAProxy:~# █
```

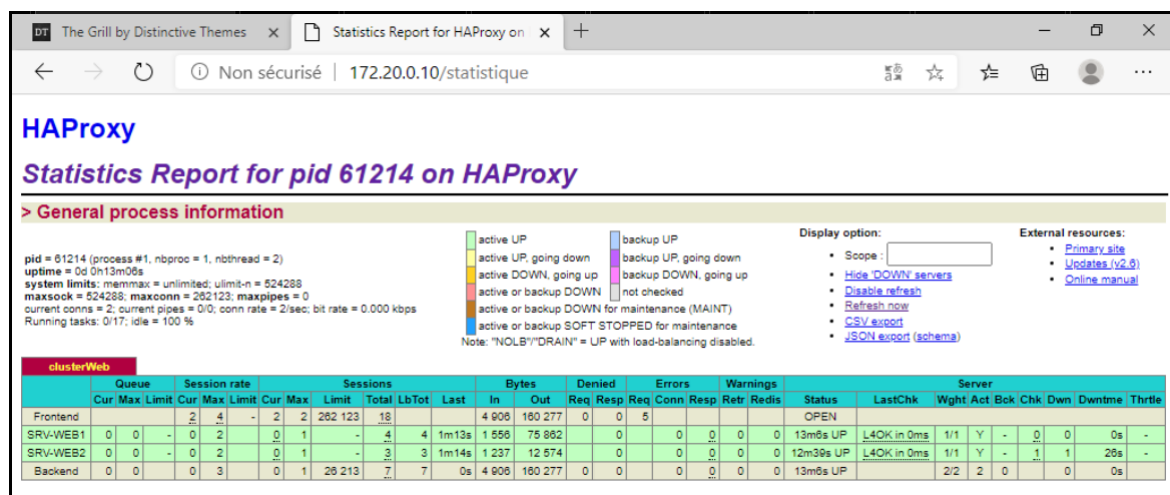
Page de statistiques HAProxy

Cette page, accessible via <http://172.20.0.10/statistique>, permet de visualiser en temps réel l'état du cluster HAProxy et de ses serveurs web associés.

On y observe que les deux serveurs — SRV-WEB1 et SRV-WEB2 — sont en ligne (UP), ce qui confirme le bon fonctionnement du load balancing.

Les colonnes indiquent notamment le nombre de connexions actives, le taux de requêtes traitées, ainsi que l'état de chaque backend.

Cette interface prouve que la répartition de charge et la haute disponibilité entre les deux serveurs sont pleinement opérationnelles, sans SPOF (Single Point of Failure).



Conclusion du projet

Grâce à la mise en place d'un deuxième serveur HAProxy, nous avons supprimé le Single Point of Failure (SPOF) présent dans l'infrastructure initiale.

Le cluster HAProxy/Heartbeat assure désormais la haute disponibilité du service web : si le premier nœud HAProxy tombe, le second prend automatiquement le relais sans interruption pour l'utilisateur.

Cette redondance garantit une continuité de service et une meilleure tolérance aux pannes, tout en maintenant la répartition de charge entre les serveurs web.

Le projet est donc abouti : le système est désormais fiable, stable et résilient, capable de fonctionner même en cas de défaillance d'un des nœuds du cluster.